به نام خدا



درس داده کاوی- ترم ۴۰۵۱

۲.۳ بخش دوم : شرح مسئله

۱.۲.۳ گام اول : پیش پردازش و پرهیز از نشت داده

۱ . زمان‌های dispatched_on (UTC)وdriver_reached_on را به وقت محلی آستین (Austin Local Time)تبدیل کنید و بر مبنای مشترک ببرید و ستون هدف on_timeرا بسازید :اگر فاصله‌شان حداکثر ۵ دقیقه باشد، یک؛ وگرنه صفر .سهم هر کلاس را گزارش دهید .

بارگذاری دیتاست:

```
import pandas as pd

Full_data = pd.read_csv("austin_commute_raw.csv")
```

```
Full_data["dispatched_on"] = pd.to_datetime(
    Full_data["dispatched_on"], utc=True, errors="coerce"
)

Full_data["driver_reached_on"] = pd.to_datetime(
    Full_data["driver_reached_on"], utc=True, errors="coerce"
)

Full_data["time_diff"] = (
    Full_data["driver_reached_on"] - Full_data["dispatched_on"]
).dt.total_seconds() / 60

Full_data["on_time"] = pd.NA

valid = Full_data["time_diff"].notna()

Full_data.loc[valid, "on_time"] = (
    Full_data.loc[valid, "time_diff"] <= 5
).astype(int)

class_counts = Full_data.loc[valid, "on_time"].value_counts().sort_index()
class_percentages = (
    Full_data.loc[valid, "on_time"].value_counts(normalize=True).sort_index() * 100
)

print("Class counts:")
print(class_counts)

print("\nClass percentages:")
print(class_percentages.round(2))

print("\nMissing target values:", Full_data["on_time"].isna().sum())
```

ابتدا ستون‌های dispatched_onوdriver_reached_on به فرمت تاریخ و زمان تبدیل شدند و هر دو بر اساس UTC یکسان‌سازی شدند .سپس اختلاف بین زمان اعزام راننده و زمان رسیدن او بر حسب دقیقه محاسبه شد .بر اساس این اختلاف، ستون هدف on_timeایجاد شد؛ به این صورت که اگر راننده حداکثر تا ۵ دقیقه بعد از زمان اعزام رسیده باشد، مقدار ۱ و در غیر این صورت مقدار صفر در نظر گرفته شد .در نهایت تعداد و درصد نمونه‌های هر کلاس محاسبه شد تا توزیع کلاس‌های هدف مشخص شود.

خروجی:

```
Class counts:
on_time
0    62849
1    87013
Name: count, dtype: int64

Class percentages:
on_time
0    41.94
1    58.06
Name: proportion, dtype: float64

Missing target values: 138
```

۲. ستون‌های year، make، model، color، requested_car_category وstatus را از حالت خام رشته‌ای پایتون (شکل 'b'...') باز کنید .ستون year را هم به عدد صحیح تبدیل کنید؛ سپس ستون‌های متنی را نرمال‌سازی کنید (فاصله‌های اضافه و املای ناهمسان مثل Chevrolet/Chevy را اصلاح کنید) و تعداد مقادیر یکتا را قبل و بعد مقایسه کنید.

```python
import pandas as pd
import ast
import re

columns = [
    "year",
    "make",
    "model",
    "color",
    "requested_car_category",
    "status"
]

unique_before = Full_data[columns].nunique(dropna=True)

def decode_bytes(value):
    if pd.isna(value):
        return value

    if isinstance(value, bytes):
        return value.decode("utf-8", errors="replace")

    if isinstance(value, str):
        value = value.strip()

        if value.startswith("b'") or value.startswith('b"'):
            try:
                decoded = ast.literal_eval(value)
                if isinstance(decoded, bytes):
                    return decoded.decode("utf-8", errors="replace")
            except (ValueError, SyntaxError):
                pass

        return value

    return value
```

```python
for col in columns:
    Full_data[col] = Full_data[col].apply(decode_bytes)


Full_data["year"] = pd.to_numeric(
    Full_data["year"], errors="coerce"
).astype("Int64")


text_columns = [
    "make",
    "model",
    "color",
    "requested_car_category",
    "status"
]

for col in text_columns:
    Full_data[col] = Full_data[col].apply(
        lambda x: re.sub(r"\s+", " ", x).strip().lower()
        if isinstance(x, str) else x
    )


make_mapping = {
    "chevy": "chevrolet",
    "mercedes benz": "mercedes-benz",
    "merceded benz": "mercedes-benz",
    "mercedes": "mercedes-benz",
    "infinity": "infiniti",
    "toytoa": "toyota",
    "volkwagen": "volkswagen",
    "licoln": "lincoln",
    "tessla": "tesla",
    "mini cooper": "mini",
    "sienna": "toyota"
}

Full_data["make"] = Full_data["make"].replace(make_mapping)


unique_after = Full_data[columns].nunique(dropna=True)

comparison = pd.DataFrame({
    "Unique Before": unique_before,
    "Unique After": unique_after,
    "Difference": unique_before - unique_after
})

print(comparison)
print("on_time" in Full_data.columns)
print(Full_data["on_time"].value_counts(dropna=False))
```

ابتدا تعداد مقادیر یکتای هر ستون قبل از انجام تغییرات محاسبه شد .سپس مقادیر باینری که به شکل b'...'ذخیره شده بودند، به رشته‌های معمولی تبدیل شدند .ستون yearنیز با استفاده از to_numericبه نوع عددی صحیح تبدیل شد و مقادیر نامعتبر آن به‌صورت مقدار گمشده در نظر گرفته شدند .در ادامه، فاصله‌های اضافی در ستون‌های متنی حذف و تمام حروف به حروف کوچک تبدیل شدند تا مقادیری که فقط از نظر نوشتاری تفاوت داشتند یکسان شوند .برای اصلاح تفاوت‌های املایی و نام‌های جایگزین در ستون makeنیز از یک دیکشنری استفاده شد؛ مثلاً مقدار chevyبه chevroletتغییر کرد .در پایان، تعداد مقادیر یکتای ستون‌ها قبل و بعد از پاک‌سازی در یک جدول قرار گرفت تا میزان تغییرات مشخص شود.

خروجی:

```
                         Unique Before  Unique After  Difference
year                                18            18           0
make                                62            40          22
model                              424           410          14
color                               32            24           8
requested_car_category               5             5           0
status                               2             2           0
True
on_time
1       87013
0       62849
<NA>      138
```

۳. درصد مقادیر گمشده‌ی هر ستون را محاسبه کنید و مقادیر ستون‌های dispatch_location_lat و dispatch_location_long را با هم مقایسه کنید؛ آیا این دو واقعاً دو کمیت متفاوت‌اند؟

```python
missing_count = Full_data.isna().sum()
missing_percent = Full_data.isna().mean() * 100

missing_summary = pd.DataFrame({
    "Missing Count": missing_count,
    "Missing Percent": missing_percent
})

missing_summary = missing_summary.sort_values(
    by="Missing Percent", ascending=False
)

print(missing_summary.round(2))


print("\nDispatch location comparison:")

print(
    missing_summary.loc[
        ["dispatch_location_lat", "dispatch_location_long"]
    ]
)
```

خروجی:

```
                              Missing Count  Missing Percent
driving_distance_to_rider             30752            20.50
driver_rating                         13896             9.26
esimtated_time_arrive                  7869             5.25
rider_rating                           2629             1.75
driving_time_to_rider                  2198             1.47
...                                     ...              ...
pickup_zone_Southeast                     0             0.00
pickup_zone_Southwest                     0             0.00
created_hour                              0             0.00
created_minute                            0             0.00
created_dayofweek                         0             0.00

[107 rows x 2 columns]
Equal values: 150000
Different values: 0
   dispatch_location_lat  dispatch_location_long
0                 -97.752                 -97.752
1                 -97.749                 -97.749
2                 -97.666                 -97.666
3                 -97.744                 -97.744
4                 -97.752                 -97.752
```

خروجی نشان داد که در هر ۱۵۰,۰۰۰ ردیف، مقدار این دو ستون کاملاً برابر است .از آنجا که این دو ستون باید به‌ترتیب عرض و طول جغرافیایی را نشان دهند، یکسان بودن تمام مقادیر می‌تواند نشان‌دهنده‌ی وجود مشکل در ثبت یا استخراج داده‌ها باشد .بنابراین، این دو ستون اطلاعات جغرافیایی متفاوتی ارائه نمی‌دهند .

۴. فهرست کنید کدام ستون‌ها اطلاعاتی را فاش می‌کنند که در لحظه‌ی dispatched_onوجود نداشته‌اند (نشت داده). حداقل ستون‌های driver_reached_on، started_on، completed_on، esimtated_time_arriveو driving_time_to_riderرا بررسی کنید و برای هرکدام یک جمله بنویسید که چرا نباید مستقیماً ویژگی مدل باشند .در پایان، فهرست نهایی ویژگی‌های مجاز برای مدل‌سازی را اعلام کنید.

```
drop_columns = [
    "RIDE_ID",
    "rider_id",
    "driver_accepted_on",
    "model",
    "driver_reached_on",
    "dispatched_on",
    "created_date",
    "started_on",
    "esimtated_time_arrive",
    "completed_on",
    "dispatch_location_long",
    "dispatch_location_lat",
    "driving_time_to_rider",
    "time_diff",
    "status"
]
categorical_cols = [
    "requested_car_category",
    "make",
    "color",
    "pickup_zone"
]
Full_data = pd.get_dummies(Full_data , columns=categorical_cols)

Full_data["created_date"] = pd.to_datetime(
    Full_data["created_date"],
    errors="coerce",
    utc=True
)
Full_data["created_hour"] = Full_data["created_date"].dt.hour
Full_data["created_minute"] = Full_data["created_date"].dt.minute
Full_data["created_dayofweek"] = Full_data["created_date"].dt.dayofweek

Final_data = Full_data.drop(
    columns=drop_columns,
    errors="ignore"
)

Final_data = Final_data.dropna(subset=["on_time"])

X = Final_data.drop(columns=["on_time"])
y = Final_data["on_time"].astype(int)

print("Final attributes:")
print(X.columns.tolist())
print("Number of attributes:", X.shape[1])
```

خروجی:

```
Final attributes:
['start_location_lat', 'start_location_long', 'driver_rating', 'rider_rating', 'surge_factor', 'base_fare', 'rate_per_mile', 'rate_per_minute', 'driving_distance
Number of attributes: 91
```

در این مرحله ستون‌هایی که اطلاعات آن‌ها در زمان اعزام راننده در دسترس نیست یا مستقیماً اطلاعات مربوط به نتیجه‌ی سفر را نشان می‌دهند، از داده‌ها حذف شدند .همچنین شناسه‌هایی مانند RIDE_IDو ستون‌هایی با تعداد مقادیر یکتای زیاد که برای مدل اولیه مناسب نیستند، کنار گذاشته شدند .از ستون created_dateنیز سه ویژگی ساعت، دقیقه و روز هفته استخراج شد تا اطلاعات زمانی آن حفظ شود .ستون‌های دسته‌ای مانند نوع خودرو، سازنده، رنگ و ناحیه‌ی مبدا نیز با روش One-Hot Encodingبه ویژگی‌های عددی تبدیل شدند .در نهایت، داده‌ها به دو بخش Xشامل ویژگی‌های ورودی و y شامل متغیر هدف تقسیم شدند.

driver_reached_on

زمان واقعی رسیدن راننده را نشان می‌دهد و بعد از زمان اعزام مشخص می‌شود؛ بنابراین مستقیماً با هدف پیش‌بینی ارتباط دارد.

started_on

زمان شروع سفر را نشان می‌دهد که پس از رسیدن راننده و شروع سفر ثبت می‌شود.

completed_on

زمان پایان سفر را نشان می‌دهد و در لحظه‌ی اعزام هنوز مشخص نیست.

esimtated_time_arrive

زمان تخمینی رسیدن راننده را نشان می‌دهد؛ اگر این مقدار در لحظه‌ی اعزام تولید نشده یا از اطلاعات بعدی محاسبه شده باشد، استفاده از آن باعث نشت داده می‌شود.

driving_time_to_rider

مدت زمان رانندگی تا مسافر را نشان می‌دهد و ممکن است بر اساس مسیر یا اطلاعاتی محاسبه شده باشد که در لحظه‌ی اعزام در دسترس نبوده‌اند.

driver_accepted_on

زمان پذیرش درخواست توسط راننده را نشان می‌دهد که بعد از زمان اعزام اتفاق می‌افتد.

time_diff

مستقیماً از اختلاف زمان رسیدن راننده و اعزام ساخته شده و اطلاعات مربوط به متغیر هدف را در خود دارد.

status

ممکن است وضعیت نهایی سفر یا لغو درخواست را نشان دهد و اطلاعاتی از اتفاقات بعد از اعزام را فاش کند.

RIDE_ID

شناسه‌ی سفر است و اطلاعات مفیدی برای پیش‌بینی زمان رسیدن راننده ندارد.

created_date

تاریخ خام است و به‌جای آن می‌توان ویژگی‌هایی مثل ساعت و روز هفته را استخراج کرد.

در خروجی قبلی مشـــخص شـــد که dispatch_location_latو dispatch_location_longدر تمام ردیف‌ها برابر هســـتند؛ بنابراین هر دو از ویژگی‌ها حذف شدند، چون اطلاعات جغرافیایی معتبری ارائه نمی‌کنند.

۳. ۲. ۲ گام دوم : دسته بندی پایه

۱. داده‌ها را به سه بخش آموزش، اعتبارسنجی و آزمون (مثلاً نسبت ۱۵/۱۵/۷۰) تقسیم کنید، به‌طوری‌که نسبت دو کلاس on_timeدر هر سه بخش حفظ شود.

```
from sklearn.model_selection import train_test_split


X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)


X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)
print("Train:")
print("Total:",len(X_train))
print(y_train.value_counts(normalize=True) * 100)

print("\nValidation:")
print("Total:",len(X_val))
print(y_val.value_counts(normalize=True) * 100)

print("\nTest:")
print("Total:",len(X_test))
print(y_test.value_counts(normalize=True) * 100)


print("Total:", len(X_train) + len(X_val) + len(X_test))
```

خروجی:

```
Train:
Total: 104903
on_time
1    58.06221
0    41.93779
Name: proportion, dtype: float64

Validation:
Total: 22479
on_time
1    58.063081
0    41.936919
Name: proportion, dtype: float64

Test:
Total: 22480
on_time
1    58.060498
0    41.939502
Name: proportion, dtype: float64
Total: 149862
```

در این مرحله داده‌ها به سه بخش آموزش، اعتبارسنجی و آزمون تقسیم شدند .ابتدا ۷۰ درصد داده‌ها برای آموزش مدل جدا شد و ۳۰ درصد باقی‌مانده به‌صورت موقت نگه داشته شد .سپس این بخش به دو قسمت مساوی تقسیم شد تا هرکدام ۱۵ درصد از کل داده‌ها را تشکیل دهند .برای حفظ نسبت کلاس‌های صفر و یک در هر سه مجموعه، از پارامتر stratifyاستفاده شد .همچنین با استفاده از value_counts(normalize=True)درصد هر کلاس در هر بخش محاسبه شد تا توزیع کلاس‌ها بررسی شود .

```
X_train["driving_distance_to_rider"] = X_train["driving_distance_to_rider"].fillna(X_train["driving_distance_to_rider"].median())
X_val["driving_distance_to_rider"] = X_val["driving_distance_to_rider"].fillna(X_train["driving_distance_to_rider"].median())
X_test["driving_distance_to_rider"] = X_test["driving_distance_to_rider"].fillna(X_train["driving_distance_to_rider"].median())


X_train["driver_rating"] = X_train["driver_rating"].fillna(X_train["driver_rating"].median())
X_val["driver_rating"] = X_val["driver_rating"].fillna(X_train["driver_rating"].median())
X_test["driver_rating"] = X_test["driver_rating"].fillna(X_train["driver_rating"].median())
```

همانطور که در قسمت های قبل مشاهده شد ، مقادیر خالی این دو ستون درصد قابل توجهی از مقادیر ستون خود را تشکیل میدادند. به همین دلیل مقادیر خالی با میانه هر ستون آن ها در داده ترین جایگزین شدند.

۲. دقت یک مدل پایه که همیشه کلاس اکثریت را پیش‌بینی می‌کند (Majority Baseline)روی هر سه بخش محاسبه کنید .از این پس، هر مدلی که ساخته می‌شود باید با این عدد مقایسه شود.

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

baseline = DummyClassifier(strategy="most_frequent")

baseline.fit(X_train, y_train)


y_train_pred = baseline.predict(X_train)
y_val_pred = baseline.predict(X_val)
y_test_pred = baseline.predict(X_test)


print("Train Baseline Accuracy:",
      round(accuracy_score(y_train, y_train_pred) , 2))

print("Validation Baseline Accuracy:",
      round(accuracy_score(y_val, y_val_pred) , 2))
print("Test Baseline Accuracy:",
      round(accuracy_score(y_test, y_test_pred) , 2))
```

✓ 0.0s

```
Train Baseline Accuracy: 0.58
Validation Baseline Accuracy: 0.58
Test Baseline Accuracy: 0.58
```

در این مرحله یک مدل پایه با استفاده از DummyClassifierساخته شد که در تمام پیش‌بینی‌ها فقط کلاس اکثریت داده‌های آموزشی را انتخاب می‌کند .این مدل هیچ الگوی خاصی از داده‌ها یاد نمی‌گیرد و صرفاً به‌عنوان یک معیار اولیه برای ارزیابی عملکرد مدل‌های اصلی استفاده می‌شود .پس از آموزش مدل روی داده‌های آموزشی، دقت آن روی هر سه مجموعه‌ی آموزش، اعتبارسنجی و آزمون محاسبه شد . برابر شدن دقت ها هم به دلیل برابر بودن نسبت دو کلاس در هر نوع دیتا ست ترین ، تست و ولیدیشن است.

۳. با ویژگی‌های مجازی که در گام اول مشخص کردید، یک درخت تصمیم با تنظیمات پیش‌فرض (بدون محدودیت عمق) آموزش دهید .دقت روی مجموعه‌ی آموزش و اعتبارسنجی را گزارش دهید.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score


dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)


y_train_pred = dt_model.predict(X_train)
y_val_pred = dt_model.predict(X_val)


train_acc = accuracy_score(y_train, y_train_pred)
val_acc = accuracy_score(y_val, y_val_pred)

print("Train Accuracy:", f"{train_acc:.2f}")
print("Validation Accuracy:", f"{val_acc:.2f}")
```

✓ 1.7s

```
Train Accuracy: 1.00
Validation Accuracy: 0.67
```

در این مرحله یک مدل درخت تصمیم با استفاده از تنظیمات پیش‌فرض و بدون محدودیت عمق ساخته شد .سپس مدل با داده‌های آموزشی آموزش داده شد و دقت آن روی مجموعه‌های آموزش و اعتبارسنجی محاسبه شد . همانطور که مشاهده میشـود درخت بدون محدودیت باعث ایجاد overfeating میشـود و مدل داده های آموزشـی رو حفظ میکند ولی روی داده های ولیدیشن عملکرد خوبی ندارد.

۴. معیار Giniیا Entropyرا که مدل برای تقسیم گره ریشه انتخاب کرده است، با دست و با استفاده از فرمول (نه با فراخوانی مستقیم تابع کتابخانه) محاسبه کنید و نتیجه را با مقداری که کتابخانه گزارش می‌دهد مقایسه کنید.

```
import numpy as np

def gini_manual(y):
    if len(y) == 0:
        return 0

    values, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)

    return 1 - np.sum(probabilities ** 2)


def entropy_manual(y):
    if len(y) == 0:
        return 0

    _values, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)

    return -np.sum(probabilities * np.log2(probabilities))


gini_manual_value = gini_manual(y_train)
entropy_manual_value = entropy_manual(y_train)


criterion = dt_model.criterion
```

```python
root_value = dt_model.tree_.impurity[0]

print("Criterion used by the tree:", criterion)
print("Library root impurity:",round(root_value , 3))

if criterion == "gini":
    print("Manual Gini:", round(gini_manual_value , 3))
    print("Difference:", abs(gini_manual_value - root_value))

elif criterion == "entropy":
    print("Manual Entropy:",round(entropy_manual_value , 3))
    print("Difference:", abs(entropy_manual_value - root_value))
```

✓ 0.0s

```
Criterion used by the tree: gini
Library root impurity: 0.487
Manual Gini: 0.487
Difference: 0.0
```

ابتدا دو تابع برای محاسبه‌ی دستی معیارهای Giniو Entropyنوشته شد .در هر دو تابع، تعداد نمونه‌های هر کلاس با استفاده از np.uniqueبه دست آمد و سپس احتمال هر کلاس با تقسیم تعداد آن بر کل نمونه‌ها محاسبه شد .برای محاسبه‌ی Giniاز فرمول $1 - \sum p_i^2$ و برای محاسبه‌ی Entropyاز فرمول $-\sum p_i \log_2(p_i)$ استفاده شد .در ادامه، معیار انتخاب‌شده توسط مدل از طریق ویژگی criterionمشخص شد و مقدار ناخالصی گره ریشه نیز از dt_model.tree_.impurity[0]دریافت شد .

۳. ۲. ۳ گام سوم : بیش برازش

۱. پنج درخت تصمیم با پنج مقدار متفاوت برای عمق)مثلاً ۲، ۴، ۸، ۱۶ و بدون محدودیت (آموزش دهید و دقت آموزش و اعتبارسنجی هرکدام را در یک جدول کنار هم قرار دهید.

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

depths = [2, 4, 8, 16, None]
results = []

for depth in depths:
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
    )

    model.fit(X_train, y_train)

    train_pred = model.predict(X_train)
    val_pred = model.predict(X_val)

    train_acc = accuracy_score(y_train, train_pred)
    val_acc = accuracy_score(y_val, val_pred)

    results.append({
        "Max Depth": depth if depth is not None else "Unlimited",
        "Train Accuracy": train_acc,
        "Validation Accuracy": val_acc
    })

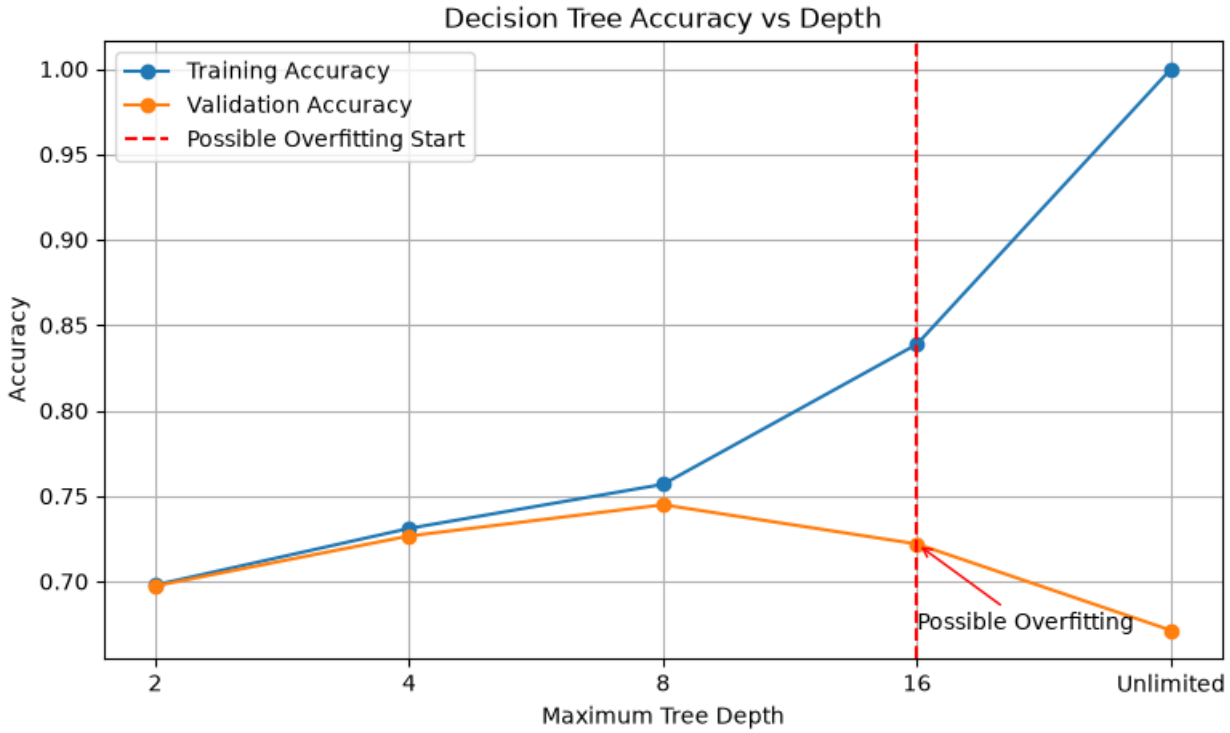
results_df = pd.DataFrame(results)
print(results_df.round(4))
```

خروجی:

	Max Depth	Train Accuracy	Validation Accuracy
0	2	0.6979	0.6974
1	4	0.7310	0.7266
2	8	0.7569	0.7449
3	16	0.8389	0.7219
4	Unlimited	1.0000	0.6713

در این مرحله پنج مدل درخت تصمیم با عمق‌های متفاوت شامل ۲، ۴، ۸، ۱۶ و بدون محدودیت آموزش داده شدند .برای هر مدل، ابتدا داده‌های آموزشی به مدل داده شد و سپس پیش‌بینی روی مجموعه‌های آموزش و اعتبارسنجی انجام گرفت .در ادامه دقت هر دو مجموعه محاسبه شد و نتایج در یک جدول قرار گرفت تا امکان مقایسه‌ی عملکرد مدل‌ها وجود داشته باشد .

۲. یک نمودار رسم کنید که محور افقی عمق درخت و محور عمودی دقت باشد .دو خط جدا برای آموزش و اعتبارسنجی رسم کنید و نقطه‌ی شروع بیش‌برازش را روی نمودار مشخص کنید.



در این نمودار عمق ۸ بهترین دقت ولیدیشن رو مشخص میکند و بعد از آن دقت ولیدیشن کم میشود که اولین نقطه ای که از آنجا به بعد اورفیت رخ می دهد عمق ۱۶ است .

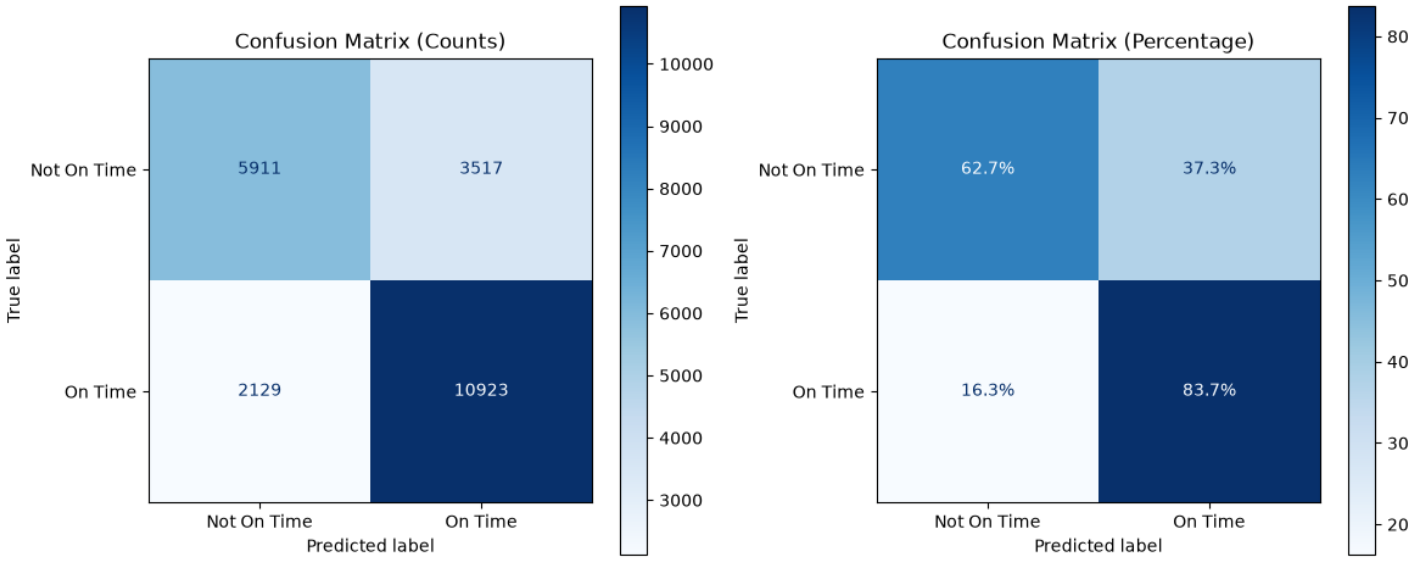
۳. یکی از این پنج مدل را به‌عنوان مدل نهایی انتخاب کنید و دلیل انتخاب را بنویسید (دلیل : بالاترین دقت روی داده‌ی آموزش پذیرفته نیست) ، و در یک پاراگراف بگویید موازنه‌ی bias/variance دقیقاً کجای این نمودار خودش را نشان می‌دهد.

مدل با عمق ۸ به‌عنوان مدل نهایی انتخاب می‌شـــود؛ زیرا در میان پنج مدل بررســـی‌شـــده، بالاترین دقت را روی مجموعه‌ی اعتبارسـنجی، با مقداری حدود ۰/۷۴، به دسـت آورده اسـت .در عمق‌های کمتر، مدل هنوز به اندازه‌ی کافی پیچیده نیسـت و دقت اعتبارســـنجی پایین‌تری دارد .از طرف دیگر، با افزایش عمق به ۱۶ و حالت بدون محدودیت، دقت آموزش افزایش پیدا می‌کند، اما دقت اعتبارسنجی کاهش می‌یابد .

موازنه‌ی Bias وVariance در این نمودار از طریق تغییرات دقت آموزش و اعتبارسـنجی با افزایش عمق درخت قابل مشـاهده اسـت .در عمق‌های پایین، مدل پیچیدگی کافی برای یادگیری الگوهای داده ندارد و با بایاس بالا مواجه اسـت .با افزایش عمق تا ۸، پیچیدگی مدل بیشـتر شـده و دقت اعتبارسـنجی افزایش می‌یابد؛ در نتیجه بایاس کاهش پیدا می‌کند .اما از عمق ۸ به بعد، دقت آموزش همچنان افزایش می‌یابد، درحالی‌که دقت اعتبارسنجی کاهش پیدا می‌کند .این رفتار نشان‌دهنده‌ی افزایش واریانس و شـروع بیش‌برازش اسـت .بنابراین، عمق ۸ نقطه‌ای اسـت که تعادل مناسـبی میان بایاس و واریانس برقرار شـده و بهترین عملکرد روی داده‌های اعتبارسنجی حاصل شده است.

۳. ۲. ۴ گام چهارم : ارزیابی مدل

۱. مدل منتخب گام ســـوم را فقط یک‌بار روی مجموعه‌ی آزمون اجرا کنید و ماتریس درهم‌ریختگی (Confusion Matrix)را رسم کنید.



نتایج ماتریس نشـان می‌دهد که مدل نهایی در مجموع ۱۶۸۳۴ پیش‌بینی صـحیح و ۵۶۴۶ پیش‌بینی اشـتباه داشـته اسـت . مدل توانسـته اسـت ۸۳/۷ درصـد از سـفرهای به‌موقع و ۶۲/۷ درصـد از سـفرهای غیربه‌موقع را به‌درسـتی شـناسـایی کند .این نتایج نشان می‌دهد که مدل در تشخیص سفرهای به‌موقع عملکرد بهتری دارد و در شناسایی سفرهای غیربه‌موقع با خطای بیشتری مواجه است .

نکته: کد نمودار های مت پلات و ماتریکس در هم ریختگی با اسـتفاده از هوش مصـنوعی CHAT GPT رسـم و انجام شـده است.

۲. معیارهای Accuracy، Precision، Recallو F1را برای هر دو کلاس محاسبه کنید.

```python
from sklearn.metrics import classification_report

accuracy = accuracy_score(y_test, y_test_pred)
print("Accuracy:", accuracy)

report = classification_report(
    y_test,
    y_test_pred,
    labels=[0, 1],
    target_names=["Not On Time", "On Time"],
    output_dict=True
)

report_df = pd.DataFrame(report).T

report_df = report_df.drop(
    ["accuracy"]
)

print(report_df.round(3))
```

✓ 0.0s

```
Accuracy: 0.7488434163701068
              precision  recall  f1-score  support
Not On Time       0.735   0.627     0.677   9428.0
On Time           0.756   0.837     0.795  13052.0
macro avg         0.746   0.732     0.736  22480.0
weighted avg      0.748   0.749     0.745  22480.0
```

۳. مشخص کنید مدل بیشتر مرتکب خطای مثبت کاذب می‌شود یا منفی کاذب، و توضیح دهید این نوع خطا برای کاربری که می‌خواهد بین اسنپ و مترو انتخاب کند چه پیامد عملی دارد.

با توجه به ماتریس درهم‌ریختگی، مدل بیشتر مرتکب خطای مثبت کاذب (False Positive)می‌شود.

- مثبت کاذب :(FP)تعداد ۳۵۱۷ مورد؛ سفرهایی که واقعاً غیربه‌موقع بوده‌اند، اما مدل آن‌ها را به‌موقع پیش‌بینی کرده است.
- منفی کاذب :(FN)تعداد ۲۱۲۹ مورد؛ سفرهایی که واقعاً به‌موقع بوده‌اند، اما مدل آن‌ها را غیربه‌موقع پیش‌بینی کرده است.

در این مسئله، مثبت کاذب به این معناست که مدل به کاربر اعلام می‌کند راننده اسنپ حداکثر تا ۵ دقیقه می‌رسد، درحالی‌که در واقع رسیدن راننده بیشتر از ۵ دقیقه طول می‌کشد. در چنین شرایطی، کاربر ممکن است با اعتماد به پیش‌بینی مدل، مترو را انتخاب نکند و منتظر راننده بماند؛ در نتیجه احتمال دارد زمان زیادی را از دست بدهد و به مقصد دیر برسد. در مقابل، منفی کاذب ممکن است باعث شود کاربر تصور کند راننده به‌موقع نمی‌رسد و مترو را انتخاب کند، درحالی‌که امکان رسیدن به‌موقع راننده وجود داشته است. با توجه به بیشتر بودن خطای مثبت کاذب، خطر اصلی مدل این است که کاربر را به انتظار برای خودرویی سوق دهد که در زمان پیش‌بینی‌شده نمی‌رسد.

۳. ۲. ۵ گام پنجم : ارزیابی جغرافیایی

۱. معیارهای Accuracy و F1-scoreرا برای مدل منتخب، به تفکیک پنج مقدار ستون pickup_zoneمحاسبه کنید.

```python
from sklearn.metrics import accuracy_score, f1_score

zones = [col for col in X_test.columns if col.startswith("pickup_zone_")]

results = []

for col in zones:
    mask = X_test[col] == 1
    y_true_zone = y_test[mask]
    y_pred_zone = y_test_pred[mask]

    acc = accuracy_score(y_true_zone, y_pred_zone)

    f1_scores = f1_score(
        y_true_zone,
        y_pred_zone,
        average=None,
        labels=[0, 1],
        zero_division=0
    )
    results.append({
        "Pickup Zone": col.replace("pickup_zone_", ""),
        "Accuracy": acc,
        "F1 Not On Time": f1_scores[0],
        "F1 On Time": f1_scores[1]
    })

results_df = pd.DataFrame(results)

print(results_df.round(4))
```

✓ 0.0s

	Pickup Zone	Accuracy	F1 Not On Time	F1 On Time
0	Downtown	0.7775	0.3967	0.8636
1	Northeast	0.7340	0.7332	0.7349
2	Northwest	0.7488	0.7570	0.7401
3	Southeast	0.6855	0.7044	0.6640
4	Southwest	0.7501	0.7579	0.7417

در این کد، عملکرد مدل منتخب به تفکیک پنج منطقه‌ی ستون pickup_zoneبررسی شده و معیارهای Accuracyو F1-scoreبرای هر دو کلاس به‌صورت جداگانه محاسبه شده‌اند.

ابتدا ستون‌های مربوط به پنج منطقه استخراج می‌شوند .سپس با استفاده از یک حلقه، نمونه‌های متعلق به هر منطقه از مجموعه‌ی آزمون جدا شده و مقادیر واقعی و پیش‌بینی‌شده‌ی مدل برای آن‌ها مشخص می‌شوند .در ادامه، Accuracy به‌عنوان معیار کلی صحت پیش‌بینی‌ها محاسبه می‌شود .همچنین با استفاده از average=None، مقدار F1-scoreبرای هر دو کلاس Not On Time و On Timeبه‌صورت مستقل به دست می‌آید .

۲. آیا عملکرد مدل در ناحیه‌ی Downtownبا سایر نواحی تفاوت معناداری دارد؟ یک فرضیه برای دلیل این تفاوت پیشنهاد دهید و توضیح دهید چرا یک عدد کلی Accuracyمی‌تواند چنین تفاوتی را پنهان کند.

نتایج نشان می‌دهد که عملکرد مدل در منطقه‌ی Downtownاز نظر Accuracyبا سایر مناطق متفاوت است .مقدار Accuracyدر این منطقه برابر با ۰/۷۷۷۵ است که از تمام مناطق دیگر بیشتر است .بااین‌حال، بررسی F1-scoreنشان

می‌دهد که عملکرد مدل در تشخیص دو کلاس در این منطقه تفاوت زیادی دارد؛ به‌طوری‌که F1-scoreکلاس On Timeبرابر با ۰٫۸۶۳۶ و برای کلاس Not On Timeبرابر با ۰٫۳۹۶۷ است.

یکی از دلایل احتمالی این تفاوت می‌تواند شرایط خاص ترافیکی و الگوی سفر در منطقه‌ی Downtownباشد .برای مثال، تراکم ترافیک، تعداد زیاد درخواست‌ها و تفاوت شرایط خیابان‌ها ممکن است باعث شود الگوی زمانی رسیدن رانندگان در این منطقه با سایر مناطق متفاوت باشد.

Accuracyفقط نسبت کل پیش‌بینی‌های صحیح به تعداد کل نمونه‌ها را نشان می‌دهد و مشخص نمی‌کند که مدل هر کلاس را با چه کیفیتی تشخیص داده است .برای مثال، در منطقه‌ی Downtownبا وجود Accuracyبرابر با ۷۷٫۷۵ درصد، مقدار F1-scoreکلاس Not On Timeتنها ۰٫۳۹۶۷ است .بنابراین، ممکن است Accuracyبالا به دلیل تشخیص مناسب یک کلاس به دست آمده باشد، درحالی‌که مدل در تشخیص کلاس دیگر عملکرد ضعیفی داشته باشد .به همین دلیل، بررسی معیارهایی مانند Precision، Recallو F1-score برای ارزیابی دقیق‌تر عملکرد مدل در هر منطقه ضروری است.

۳. یادآوری کنید که downtown_kmو pickup_zoneتقسیم‌بندی رسمی RideAustinنیستند و این محدودیت را در تفسیر نتیجه در نظر بگیرید.

ستون‌های downtown_kmو pickup_zoneلزوماً نشان‌دهنده‌ی تقسیم‌بندی رسمی و تأییدشده‌ی شرکت RideAustin نیستند . بنابراین، نتایج به‌دست‌آمده از مقایسه‌ی مناطق باید با احتیاط تفسیر شوند و نمی‌توان آن‌ها را به‌عنوان تفاوت قطعی در عملکرد سرویس RideAustinدر مناطق جغرافیایی مختلف در نظر گرفت.

برای مثال، بالاتر بودن Accuracyدر منطقه‌ی Downtownلزوماً به معنای بهتر بودن عملکرد رانندگان یا شرایط حمل‌ونقل در مرکز شهر نیست .این تفاوت ممکن است تحت تأثیر نحوه‌ی تعریف مناطق، نحوه‌ی دسته‌بندی داده‌ها، تعداد نمونه‌ها و توزیع کلاس‌های هدف در هر منطقه باشد.

۳. ۲. ۶ گام ششم : تصمیم نهایی

با توجه به دقت و Recallمدل نهایی و تفاوتی که در گام پنجم بین نواحی جغرافیایی مشاهده کردید، آیا این مدل برای این‌که به‌جای مترو، سفر با اسنپ را به یک کاربر واقعی پیشنهاد دهد، کافی است؟ در یک یا دو پاراگراف مستدل، با ارجاع مستقیم به اعدادی که خودتان در گام‌های قبل به دست آوردید، پاسخ دهید.

با توجه به نتایج به‌دست‌آمده، مدل نهایی با عمق ۸ برای پیشنهاد سفر با اسنپ به کاربران عملکرد قابل قبولی دارد، اما نمی‌توان آن را به‌طور کامل قابل اعتماد دانست .دقت کلی مدل برابر با ٪۷۴٫۹ و Recallکلاس On Timeبرابر با ٪۸۳٫۷ است .این مقدار نشان می‌دهد که مدل توانسته است حدود ۸۴ درصد از سفرهایی را که واقعاً در کمتر از ۵ دقیقه به راننده می‌رسند، به‌درستی شناسایی کند .بااین‌حال، Precisionاین کلاس برابر با ٪۷۵٫۶ است؛ یعنی از میان پیش‌بینی‌هایی که مدل به‌عنوان سفر به‌موقع اعلام کرده، حدود ۲۴٫۴ درصد در واقع به‌موقع نبوده‌اند.

این مسئله در انتخاب بین اسنپ و مترو اهمیت زیادی دارد؛ زیرا پیش‌بینی اشتباه به‌موقع بودن سفر می‌تواند باعث شود کاربر مترو را کنار بگذارد و منتظر راننده بماند، درحالی‌که راننده در زمان مورد انتظار نمی‌رسد .همچنین بررسی مناطق جغرافیایی نشان می‌دهد که عملکرد مدل در همه‌ی مناطق یکسان نیست؛ برای مثال، در Downtownمقدار F1کلاس On Timeبرابر با ۰٫۸۶۳۶، اما F1کلاس Not On Timeتنها ۰٫۳۹۶۷ است .بنابراین، با وجود عملکرد نسبتاً مناسب مدل در

تشخیص سفرهای به‌موقع، میزان خطا و تفاوت عملکرد در مناطق مختلف نشان می‌دهد که این مدل به‌تنهایی برای ارائه‌ی پیشنهاد قطعی به کاربران کافی نیست و بهتر است پیش‌بینی آن همراه با اطلاعاتی مانند شرایط ترافیکی و زمان تخمینی رسیدن راننده در اختیار کاربر قرار گیرد.